

Algorithms Analysis (CS216)

2022-2023

Course Project (Group-based Programming Project). [15 marks]

1. Form a group of 2-3 students, and send your group details (Name in Arabic, academic No., email address)
2. Read and understand the problem stated below.
3. Design an algorithm to solve the stated problem and write its pseudocode. [5 marks]
4. Analyze your algorithm running time, and provide an asymptotic upper bound using Big-Oh notation. [4 marks]
5. Implement your algorithm using Java. [3 marks]
6. Submit your report (pseudocode, running time and asymptotic upper bound analysis) along with your .java file via Blackboard before the deadline. [3 marks]

The Snake and Ladder Problem:

Sarah takes out his **Snakes and Ladders Game**, stares the board and wonders: "If I can always roll the die to whatever number I want, what would be the **least number of rolls to reach the destination?**"

RULES:

- The game is played with cubic dice of **6** faces numbered from **1** to **A**.
- Starting from **1**, land on square **100** with the exact roll of the die. If moving the number rolled would place the player beyond square **100**, no move is made.
- If a player lands at the base of a ladder, the player must climb the ladder. Ladders go up only.
- If a player lands at the mouth of a snake, the player must go down the snake and come out through the tail. Snakes go down only.

BOARD DESCRIPTION:

- The board is always 10 x 10 with squares numbered from **1** to **100**.
- The board contains **N** ladders given in a form of 2D matrix **A** of size $N * 2$ where (**A[i][0]**, **A[i][1]**) denotes a ladder that has its base on square **A[i][0]** and end at square **A[i][1]**.
- The board contains **M** snakes given in a form of 2D matrix **B** of size $M * 2$ where (**B[i][0]**, **B[i][1]**) denotes a snake that has its mouth on square **B[i][0]** and tail at square **B[i][1]**.

Problem Constraints

$1 \leq N, M \leq 15$

$1 \leq A[i][0], A[i][1], B[i][0], B[i][1] \leq 100$

$A[i][0] < A[i][1]$ and $B[i][0] > B[i][1]$

Neither square **1** nor square **100** will be the starting point of a ladder or snake.

A square will have at most one endpoint from either a snake or a ladder.

Input Format

First argument is a 2D matrix **A** of size $N * 2$ where **(A[i][0], A[i][1])** denotes a ladder that has its base on square **A[i][0]** and end at square **A[i][1]**.

Second argument is a 2D matrix **B** of size $M * 2$ where **(B[i][0], B[i][1])** denotes a snake that has its mouth on square **B[i][0]** and tail at square **B[i][1]**.

Output Format

Return the **least number of rolls** to move from start to finish on a separate line. If there is no solution, return **-1**.

Example Input

Input 1:

```
A = [ [32, 62]
      [42, 68]
      [12, 98] ]
B = [ [95, 13]
      [97, 25]
      [93, 37]
      [79, 27]
      [75, 19]
      [49, 47]
      [67, 17]
```

Input 2:

```
A = [ [8, 52]
      [6, 80]
      [26, 42]
      [2, 72]]
```

B = [[51, 19]
[39, 11]
[37, 29]
[81, 3]
[59, 5]
[79, 23]
[53, 7]
[43, 33]
[77, 21]

Example Output

Output 1:

3

Output 2:

5

Example Explanation

Explanation 1:

The player can roll a **5** and a **6** to land at square **12**. There is a ladder to square **98**. A roll of **2** ends the traverse in **3** rolls.

Explanation 2:

The player first rolls **5** and climbs the ladder to square **80**. Three rolls of **6** get to square **98**. A final roll of **2** lands on the target square in **5** total rolls.